

Scheduling Techniques of AI Models on Modern Heterogeneous Edge GPU—A Critical Review

Ashiyana Abdul Majeed , Mahmoud Meribout , *Senior Member, IEEE*, and Safa Mohammed Sali

Abstract—In recent years, the development of specialized edge computing devices has significantly increased, driven by the growing demand for artificial intelligence (AI) models. These devices, such as the NVIDIA Jetson series, must efficiently handle increased data processing and storage requirements. However, despite these advancements, there remains a lack of frameworks that automate the optimal execution of the deep neural network (DNN). Therefore, efforts have been made to create schedulers that can manage complex data processing needs while ensuring the efficient utilization of all available accelerators within these devices, including the CPU, GPU, deep learning accelerator (DLA), programmable vision accelerator (PVA), and video image compositor (VIC). Such schedulers would maximize the performance of edge computing systems, which is crucial in resource-constrained environments. This article aims to comprehensively review the various DNN schedulers implemented on NVIDIA Jetson devices. It examines their methodologies, performance, and effectiveness in addressing the demands of modern AI workloads. By analyzing these schedulers, this review highlights the current state of the research in the field. It identifies future research and development areas, further enhancing edge computing devices' capabilities.

Index Terms—Accelerator, deep learning accelerator (DLA), neural network, performance, scheduler.

I. INTRODUCTION

AS AI-SYSTEM-ON-CHIP becomes increasingly popular and widely adopted in various embedded devices, it is vital to allocate the tasks in a way that ensures high performance while operating under low power, particularly in edge devices [1]. One such powerful edge device is the NVIDIA Jetson series, which contains hardware accelerators that are well adapted for AI and graphics applications. They comprise several hardware engines dedicated to various parallel computation models and interfaced with high-speed and low-power synchronous dynamic random access memory (SDRAM). However, in most cases, these devices balance heavy workloads, including multiple deep neural networks (DNNs), that can impair their performance. Some works consider optimizing the DNN execution in their GPUs,

such as [2], but often overlook other accelerators, leading to sub-optimal performance. Proper task scheduling and partitioning into their hardware engines can significantly enhance efficiency, thereby using the full potential of edge computing. Building schedulers remains challenging due to the heterogeneous nature of their hardware architecture and the unpredictable memory contention between different hardware engines. The contention arises from using a shared bus that connects all hardware engines to the shared SDRAM. Hence, extensive work is necessary to develop these devices' scheduling and hardware partitioning algorithms.

With the rising demand for Jetson devices, it is of the utmost importance that a framework is developed to guarantee efficient and effective performance. For instance, the Jetson series has been used in autonomous driving. Xu et al. [3] evaluate the use of the Xavier and Orin devices for pedestrian detection using an MM-Net model. Another example is a delivery robot that uses the Xavier device to navigate and reach the destination. Here, a modified single-shot detector (SSD) is utilized for object detection [4]. Apart from robotics, the Jetson series has also been utilized for monitoring purposes, particularly in oceanography. Employing such devices allows researchers to automate the process safely and more economically, as opposed to traditional methods of physically measuring the data. It also gives them access to real-time data and insights [5]. Moreover, it has also been used in industrial inspection to recognize defects [6].

Implementing a scheduler would offer a more efficient, better-utilized system in each application. Ideally, the schedulers maximize the throughput while minimizing latency and power consumption. However, satisfying these goals simultaneously is not trivial, so most algorithms tend to optimize only a few of them, while keeping the remaining below a threshold value. Taking the case of the delivery robot [4], implementing a heterogeneity-aware execution of concurrent deep neural network (HaX-CoNN) scheduler would enable multiple DNNs, each focused on different tasks to be concurrently executed. For instance, the SSD was trained to identify people, so another DNN like ResNet can detect the other obstacles. Allocating specific roles to the DNN can improve the system's performance rather than using a single, more generalized model. Moreover, using the HaX-CoNN can improve latency by 20% over GPU-only execution, thereby increasing the efficiency of the system [7]. An alternative scheduler that could be utilized is the computational parallelization for convolutional neural network (CP-CNN) scheduler, which allows the concurrent processing of frames for a single DNN model. CP-CNN can enhance the speed and efficiency of the

Received 9 July 2025; revised 30 October 2025; accepted 4 December 2025. Date of publication 16 January 2026; date of current version 6 April 2026. Paper no. TII-25-4607. (Corresponding author: Ashiyana Abdul Majeed.)

The authors are with the Department of Computer and Information Engineering, Khalifa University, Abu Dhabi, UAE (e-mail: 100059454@ku.ac.ae; mahmoud.meribout@ku.ac.ae; safa.msali@ku.ac.ae).

Digital Object Identifier 10.1109/TII.2025.3642530

SSD model by 75.6% and 75.9%, respectively [8]. As AI models increase in complexity, it is vital to prioritize energy and latency in their performance. In fact, with the rise in the use of large language models (LLMs), there has been a renewed interest in the energy-efficient execution of these models on the edge [9].

To date, no review paper has comprehensively compared the various schedulers for the recent NVIDIA Jetson edge accelerators. In [10], a survey examined the hardware and algorithmic optimizations for the old Jetson models, namely, the Jetson TK1, TX1, and TX2, where a deep learning accelerator (DLA) is absent. These algorithmic optimizations were limited to modifying the characteristics of the DNN used, including reducing the size of the DNN, image resolution, number of frames, and depth-map levels. In addition, the same survey explored works that did consider leveraging both the GPU and CPU present in the Jetson devices to improve the performance of the DNN models, covering similar issues to those discussed in this article, such as memory contention. Another review paper that dived into scheduling is [11], which considers training and inference workload scheduling for GPU data centers. It is similar to edge computing in terms of accelerator heterogeneity, as it contains CPU and GPU resources. Furthermore, there is also a resemblance in the approaches used for deep learning model training, where linear programming (LP) and performance modeling are popular in scheduling training jobs. This article seeks to systematically compare various schedulers and their methodologies targeting more recent NVIDIA edge devices while also identifying areas for potential development.

II. BACKGROUND

A. Selection Criteria

The papers were primarily conference and journal papers published in ACM and IEEE publications. Publications available on Google Scholar that contained the terms “NVIDIA Jetson AGX Orin” or “NVIDIA Jetson AGX Xavier,” “DNN” or “CNN,” “deep learning accelerator (DLA),” and “scheduling” were included in the review. Papers discussing hybrid edge-cloud systems were excluded from the review. While schedulers have been developed for other systems like the Google Coral devices [12], they are not truly heterogeneous as the GPU is used for graphical tasks, not for DNN inference. In the case of the Libre devices [13], scheduling algorithms have not been developed as of now. Moreover, these devices target lightweight applications. Therefore, the Jetson devices were chosen as a representative hardware platform for this review, as they embody the defining characteristics of modern heterogeneous architectures, comprising of the CPU, GPU, and dedicated AI accelerators. Focusing on a single representative platform ensures a fair comparison of schedulers on a consistent hardware baseline. In addition, the scheduling techniques discussed in this review can be extended to other comparable hardware platforms. Thus, the insights derived from these studies on the Jetson devices are indicative of broader scheduling behaviors in heterogeneous edge systems.

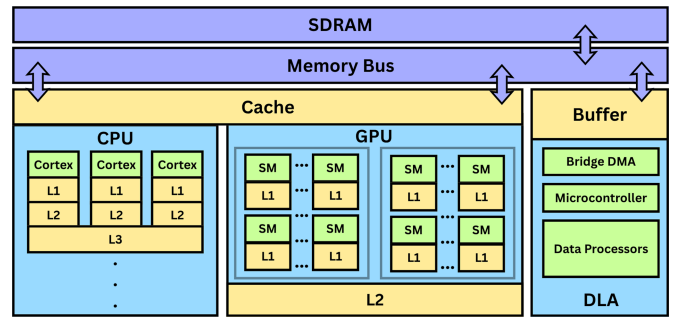


Fig. 1. Block diagram of the NVIDIA Jetson devices.

B. Hardware Architecture of the Jetson AGX Devices

The NVIDIA Jetson AGX devices (see Fig. 1) follow the general architecture of heterogeneous edge GPUs, consisting of the CPU, GPU, DLA (AI accelerator), and additional processors [like programmable vision accelerator (PVA) and video image compositor (VIC)]. Compared to Xavier, the NVIDIA AGX Orin can provide almost eight times the trillion operations per second (TOPS) for AI and deep learning applications. For instance, using PeopleNet, the Orin device can produce 536 inferences per second, whereas Xavier can only output 196 inferences in the same period [14].

1) **CPU:** The Orin device consists of three sets of four Cortex-A78 chips [14] instead of a Carmel CPU present in Xavier [15]. Apart from the three levels of cache, the system also includes a 4-MB system cache. The maximum operating frequency is 2.2 GHz [14]. It communicates with the memory using the system coherency fabric [15].

2) **GPU:** The Orin device uses an Ampere GPU [14] (rather than Volta in Xavier [15]), operating at a maximum frequency of 1.3 GHz. There are 16 streaming multiprocessors (SMs) and two levels of cache present in the device [14]. It is connected to the memory using the memory subsystem interface. The GPU module alone can yield up to 170 Sparse TOPS to run AI models [15].

3) **DLA:** The DLA is essentially a fixed-function accelerator whose computational power is not comparable to GPU/CUDA but offers significantly higher energy efficiency. In total, 2 MB of SDRAM is dedicated for use by the DLA [16]. The Orin device houses a more recent version of the DLA than the Xavier device, increasing the performance by a factor of 9. This improvement is attributed to its architecture, where a local buffer is introduced, and SDRAM bandwidth is reduced. The DLA houses specialized units, such as data processors, convolution cores, reshape engines, and additional processing elements [14].

Apart from these hardware elements, the Jetson devices integrate two additional accelerators: the PVA and VIC. The PVA consists of a Cortex-R5 chip, two direct memory access engines, and two very long instruction word vector processing units. The Orin version includes 1 MB of L2 memory cache [14]. It can handle relatively light postprocessing tasks, such as feature detection, stereo disparity, feature tracking, and object tracking [15]. VIC is responsible for preprocessing techniques, such as lens distortion correction, temporal noise reduction, and

pixel processing [15], operating up to 729.6 MHz on Orin and 1036.8 MHz on Xavier [17]. To our knowledge, no paper has used the PVA and VIC modules when scheduling tasks. Only the work in [18] models the energy and memory contention between the CPU, GPU, and PVA, and its approach could be extended to aid the scheduling algorithm in selecting the best execution pipeline. The PVA could be used for convolution layers, and the VIC could tackle preprocessing steps.

C. Software Development Kits and Libraries

NVIDIA provides resources allowing users to access the different modules in the Jetson devices and execute tasks on the desired module. CUDA [19] and cuBLAS [20] libraries are still necessary for more complex parallelism in the multicore system.

1) *TensorRT*: It allows users to implement trained DNN models on the NVIDIA Jetson devices by building an optimized inference engine. It enables users to alter techniques used in execution, such as mixed precision, and the option to choose the device for execution: GPU or DLA. TensorRT also offers interlayer and intralayer optimization techniques to ensure the best engine performance [21].

2) *DeepStream*: It is built on top of other libraries, like TensorRT, to assist in developing AI applications. It can construct a complete setup from input to output for use in computer vision applications [22].

3) *Vision Programming Interface (VPI)*: It is a library that enables the use of image and video processing algorithms, allowing access to PVA and VIC. Some examples include image conversion, filtering, lens distortion correction, and feature tracking. It is part of the DeepStream software development kit (SDK) [23].

When executing tasks incompatible with the accelerator, the aforementioned resources redirect the task to be executed on the GPU. This can lead to interruptions if a different task already occupies the GPU. Furthermore, TensorRT does not allow for DNN convolutional layers to be assigned to the PVA, limiting the use of the PVA for filtering and other preprocessing tasks. Other libraries, such as the OpenVINO, Tensorflow, and ONNX, may also be used to implement DNN models on the NVIDIA Jetson edge devices. However, they come with a disadvantage as the DLA requires a TensorRT engine plan to utilize the device.

D. Challenges of Building a Scheduler

When building a scheduler, certain challenges need to be addressed, which are listed as follows.

1) *Memory Contention*: Memory contention occurs not only in the SDRAM and secondary memory but also within the cache and between the accelerators. As shown in Fig. 1, the GPU and specialized accelerators all share a common memory bus, the memory controller fabric, connected to an external DDR5 SDRAM. Such a system necessitates careful management to ensure that no conflict arises from accelerators simultaneously accessing this memory [7], [24]. A similar problem of a single memory bus is also present in edge tensor processing unit (TPU). The elements must wait if parameters are not cached, increasing

memory traffic and power consumption. When executing AI models, almost 50.3% of energy is spent on off-chip traffic [25]. Effective scheduling must account for these potential bottlenecks to optimize overall system performance.

2) *Transition Cost*: When allocating layers to the appropriate accelerators, it is also vital to consider the possible latency incurred if consecutive layers are assigned to different devices. Data from the previous accelerator's memory must be transferred to the shared memory for the next accelerator to access the data. Hence, a latency is introduced, impacting the overall performance [7]. As such, running the consecutive layer on the same device in certain stages of the DNN may be more effective in avoiding the increasing latency. Thus, a tradeoff needs to be made between the total transition time and the number of transitions that can occur to avoid compromising the performance of the DNN model.

3) *Accelerator Properties*: Each accelerator has specific limitations concerning the tasks or layers it can handle and the input/output formats it can accept. For example, depending on the function, the PVA and VIC are restricted to particular formats. The PVA can implement convolution and filter operations on 8- and 16-bit unsigned integer formats. In contrast, the VIC can implement image conversions and other manipulations on the same formats, usually on Luma versions. The PVA's performance is close to or even faster than the CPU's for convolution. For instance, for unsigned integer 8-bit data of size 1920×1080 and a kernel of size 3, the PVA takes 0.27 ms, whereas the CPU takes 0.297 ms [23]. The performance of VIC is faster than the CPU for rescaling and some remapping tasks. In rescaling a 640×480 image to 1920×1080 , the VIC takes around 0.893 ms compared to the CPU, which takes about 6.75 ms [23]. When looking at the DLA in particular, similar constraints are present [21]. The DLA only supports FP16 and INT8 formats, with certain operations restricted to one of these formats. For example, the equal operation is supported by INT8 alone. With DNN layers, kernel sizes must range from 1 to 32, and the padding in deconvolution layers must be zero. In most cases, to ensure that the DLA can execute the portion of the DNN model, most schedulers [8], [26], [27] will allocate the beginning of the DNN model to the DLA as these layers tend to be the most compatible rather than the end of the DNN model. Hence, it is crucial to carefully consider layer specifications, input format, output format, and hardware limitations when assigning each layer to a device. This approach ensures compatibility and maximizes processing efficiency.

4) *Static Versus Dynamic Schedulers*: Schedulers can be distinguished based on parallelization techniques as well as scheduling methods. Static schedulers follow a predetermined plan, whereas dynamic schedulers actively adjust the execution strategy at runtime based on the performance metrics or the specific objectives defined by the algorithm. The aim could be minimizing latency, maximizing throughput, or reducing energy consumption, depending on the scheduler. While static schedulers are more straightforward to implement, they are inflexible when sudden changes occur during the execution pipeline. In such cases, dynamic schedulers are preferred. Almost all

the schedulers referenced in this article are static, except dynamic heterogeneity-aware execution of concurrent deep neural networks (D-HaX-CoNN) [7] and heterogeneous model to heterogeneous system mapping (H2H) [28]. In the case of edge AI applications, most have a deterministic pipeline that executes continuously. Therefore, a static scheduler is sufficient in most applications. Dynamic schedulers can be implemented in the few cases where the pipeline is unspecified, particularly when different models are required for various phases, like [29].

5) Horizontal Versus Vertical Schedulers: In most cases, schedulers divide the layers in the DNN model into subgroups to be executed on the designated accelerator, called vertical pipelining. This can be ideal for the parallel processing of video frames and for multi-DNN execution. On the other hand, few schedulers consider dividing the DNN based on the width dimension so that each accelerator can independently execute a subset of the DNN model. This type of execution could be preferable for cases with a single DNN model.

E. Metrics

The performance of schedulers is assessed using three key metrics: throughput (T), latency (L), and energy (E). The throughput refers to the amount of data a DNN model can process within a specific time frame, while latency refers to the time required to process a single input. To aid in identifying the ideal schedule, utilization may be used as it evaluates the extent to which the processing power of the available accelerators is being employed. Various other values have been used in scheduling problem formulation, as defined as follows.

- 1) a_i represents a single task from a set A of all three DNN tasks and three recurrent neural network (RNN) tasks present in the Apollo autonomous driving software [30] (from [31]).
- 2) ce_j represents a processing element from a set CE of all possible processing elements, which include two, four, or six CPU cores, four or eight GPU cores, and finally one or two DLA units (from [31]).
- 3) $L(a_i, ce_j)$ represents the latency of a task a_i executed on the element ce_j (from [31]).
- 4) $B(a_i, ce_j)$ represents the binary decision as to whether or not the task a_i is executed on the element ce_j (from [31]).
- 5) N represents a DNN model, and N_n represents a layer n present in a DNN N (from [7] and [26]).
- 6) $s(N_n)$ represents the hardware element on which the layer N_n is executed (from [7] and [26]).
- 7) $L(N_n, s(N_n))$ represents the latency of a layer N_n executed on the hardware element $s(N_n)$ (from [7] and [26]).
- 8) TR_n represents the binary decision on whether a transition occurs at this layer N_n (from [7] and [26]).
- 9) $\tau(N_n, s(N_n), \text{OUT})$ and $\tau(N_{n+1}, s(N_{n+1}), \text{IN})$ represent the transition cost (time) for the layer N_n to exit the device $s(N_n)$ and for the layer N_{n+1} to enter the device $s(N_{n+1})$ (from [7] and [26]).

- 10) $\text{pipeline}(N_n, s(N_n))$ represents the cost involved when the subunit executing the layer N_n is not the same as the previous layer N_{n+1} (from [26]).
- 11) $e(N_n, s(N_n))$ represents the energy consumed when executing layer N_n on the device $s(N_n)$. Similarly, the $e(N_n, s(N_n), \text{OUT})$ and $e(N_{n+1}, s(N_{n+1}), \text{IN})$ represent energy consumed for the layer N_n to exit the device $s(N_n)$ and for the layer N_{n+1} to enter the device $s(N_{n+1})$ (from [26]).
- 12) $C_{N_n, s(N_n)}$ represents the slowdown on layer N_n resulting from contention during the concurrent execution of layers (from [7]).
- 13) ECT represents user's energy consumption target (from [26] and [32]).
- 14) LT represents user's latency target (from [32]).
- 15) P represents user's performance objective (from [32]).
- 16) $\text{size}(F, I)$ represents memory required for storing the immediate features from each inference stage, F , and the indicator matrix, I , which states whether these features are required in the following stage (from [32]).
- 17) M represents the maximum shared memory available to the processing elements (from [32]).

These variables used in problem formulation have been curated from schedulers discussed in the upcoming sections. These were directly adopted from corresponding references.

III. SCHEDULERS

Schedulers developed for the NVIDIA edge devices can be categorized based on the methodology they use to find the optimal pipeline. They either use an optimization technique or a heuristic method to arrive at the solution. Heuristic schedulers reach the near-optimal configuration through an algorithm to find the best partitioning solution using a set of metrics that could involve latency, throughput, and power consumption. They depend on intuitive or experience-driven strategies to model the problem. As such, the optimal solution may only approximate the final schedule. On the other hand, optimization methods rely on formal techniques, such as LP and satisfiability problem solvers, to determine the optimal schedule that fulfills the objective function. The constraints emulate the limitations of the device and the application. Most of the schedulers have been developed for the Xavier device. Since both the Xavier and Orin devices are comparable architecturally, the schedulers are expected to function similarly.

A. Heuristic-Based Scheduling Techniques

1) Jetson-Aware Embedded Deep Learning Inference (Jedi): In Jedi [27], the solution is finalized when the algorithm reaches the highest throughput while maintaining a satisfactory utilization. This is done through a heuristic global search first, where a sample cut point set allows the algorithm to locate an estimated cut point, followed by a heuristic local search to determine the exact location of the cut point. In addition, experiments were conducted to attain the optimal parameter combination for increasing throughput. The first parameter is the number of threads for the operation of preprocessing and

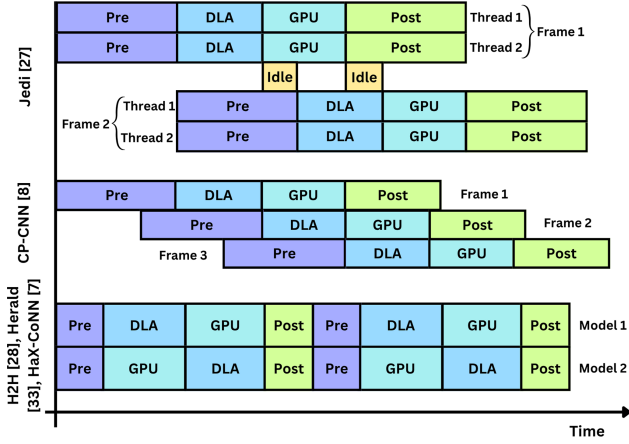


Fig. 2. Timing diagram of Jedi, CP-CNN, and multiple model schedulers (includes H2H, Herald, and HaX-CoNN).

postprocessing tasks, where they are allowed to run concurrently to prevent bottlenecks from occurring. The number of threads ranges from one to six. The second parameter is the number of streams of the assigned kernel that can be run concurrently without performance saturation occurring. Using the TensorRT framework, the number of streams ranges from one to six. The third parameter is the number of stages the pipeline can be divided into for running on different devices. This can be GPU-only, DLA-GPU, or GPU-DLA-GPU. The fourth and final parameter is the application of partial network duplication (PND), where the network portion is duplicated on the two DLA units and executed concurrently in the same stage, increasing the parallelism and utilization of the accelerators.

These parameters are implemented in the overall pipeline to produce an execution timeline shown by Fig. 2. In most cases, the number of streams for preprocessing varies from one to six, and for postprocessing, it ranges from one to three. The pipeline used the DLA with GPU in most cases, along with PND to use both the DLA units. A GPU alone could infer smaller models, like the tiny version of the you only look once (YOLO) model in integer 8-bit format. Within the accelerator, the DLA implemented four streams, while the GPU implemented two.

2) CP-CNN: CP-CNN [8] uses a different approach, where the latency and computing power are the deciding factors in finding the partitioning point. The objective of CP-CNN is to ensure that the execution time in the GPU is equivalent to the execution time in the DLA, as shown in Fig. 2. The latency of the set of frames is reduced under CP-CNN with the implementation of the DLA and the concurrent processing. Since the DLA latency and GPU latency are equal, at no point in time are the DLA and GPU left idle.

The CP-CNN algorithm iteratively compares the ratio of operations from the first layer to the current layer to the total operations in all layers (operation ratio) with the ratio of the computing power of the DLA to the total computing power (computing power ratio). The partitioning point is set as the current layer if the operation ratio is less than or equal to the computing power ratio. The algorithm also simultaneously compares the total time (operation time on DLA in addition to the data transfer time) required to execute these layers. If the latency to run till the partitioning point is more than the latency

to run till the partitioning point with one additional layer, the partitioning point is incremented by one [8].

3) Herald and H2H: Much like CP-CNN and Jedi algorithms, Herald [33] divides the layers and assigns them to the processing elements but also considers the concurrent execution of DNNs. It implements a switching technique between the DLA and GPU execution, as depicted in Fig. 2, which results in a swifter pipeline for the concurrent execution of DNNs.

A design-space exploration algorithm is implemented to assign layers based on data flow, following which postprocessing is done to eliminate idle time. The H2H [28] builds on Herald while including transition costs in the layer assignment process. After performance modeling, the mapping algorithm follows four steps to complete the scheduling. First, an initial mapping is generated based on assessing the layerwise computation, while ignoring all other factors. This mapping buffers weights into the device's local SDRAM using a Knapsack algorithm. Adjacent layers are merged to further minimize the latency incurred from the data transfer of input/output feature maps. Finally, the scheduler tries to compromise between transition and computation costs, resulting in a more transition-cost-aware scheme. The dynamic version of the H2H scheduler uses a modified Knapsack algorithm [28].

B. Optimization-Based Scheduling Techniques

1) LP: In contrast to most methods listed [1], [7], [8], [26], [27], LP methodology [31] maps the entire DNN instance onto a processing element, which can either refer to the subunits or the whole accelerator (CPU, GPU, or DLA). Since the DNN as a whole is executed on the processing element, transition points between layers are not considered. Within the multicore CPU module, two, four, or six cores are considered using the OpenMP library [34]. For the GPU module, four or eight SMs are considered using the CUDA and cuBLAS libraries [19], [20]. Using TensorRT [21], one or two units of the DLA are considered. The LP scheduler is built for use with the Apollo Autonomous Driving software [30], consisting of perception, speech recognition, trajectory planning, localization, mapping, prediction, and control modules. The perception module has four DNNs for different object detection and tracking purposes. One of them is excluded from scheduling as it requires more resources than what is available from the Xavier device. The speech recognition module also uses a DNN. The prediction module has three RNN models for lane sequence computation. These DNNs and RNNs are considered for scheduling and were evaluated to check for compatibility with the accelerators, which clarifies that RNNs cannot be executed on DLA, and some DNNs cannot be executed on the CPU. The DNNs and RNNs are executed simultaneously in the Xavier device, where each neural network instance is executed recurrently based on a defined frame rate, making the scheduler static. The objective function of the LP scheduler focuses on minimizing the cumulative latency of the system (1).

$$\min \sum_{a_i \in A, ce_j \in CE} L(a_i, ce_j) \cdot B(a_i, ce_j). \quad (1)$$

The latency of each task was found through profiling on each device [31]. Memory contention was considered an assumption

with a time factor added to the profiled timings. It does not consider the system's energy consumption. The constraints include ensuring that a task a_i can only be mapped to a single processing element ce_j and that when a ce_j is chosen, some of the other values in the set CE become obsolete. For example, if both the DLA units are chosen for a single DNN instance, the individual DLA options are no longer considered for the remaining a_i present in A . Similarly, in the case of GPU, if all eight SMs are considered for a particular model, the four-SM option is no longer present in the set CE . Rather than presenting the throughput of the system during the concurrent processing, the scheduler is assessed on its success rate (feasibility ratio) of executing the DNNs and RNNs on the set of CE . The results demonstrated that adding more devices to the set CE increases the feasibility ratio of scheduling increased workloads [31].

2) Energy-Aware Execution of Neural Network (AxoNN): The AxoNN scheduler [26] follows a similar methodology to CP-CNN, dividing layers into DLA then GPU. The AxoNN uses a constraint-based optimization problem focused on satisfiability along with empirical modeling. The energy, latency, pipeline, and transition times are found through performance modeling. A Z3 satisfiability modulo (SMT) solver is utilized to identify the ideal pipeline. The same solver has also been used in the HaX-CoNN scheduler for a satisfiability problem [7]. The objective function (2) aims to minimize the latency while ensuring that the energy consumed is below the threshold value set by the user. Equations (3) and (4) show how these values are calculated [26].

$$\begin{aligned} \min \text{latency}(N, CE, S(N \rightarrow CE)) \\ \text{s.t. energy}(N, CE, S(N \rightarrow CE)) < \text{ECT} \end{aligned} \quad (2)$$

$$\begin{aligned} \text{latency} = \sum_{n=0}^{\text{len}(N)} (L(N_n, s(N_n)) + TR_n \cdot \tau(N_n, s(N_n), \text{OUT}) \\ + TR_n \cdot \tau(N_{n+1}, s(N_{n+1}), \text{IN}) \\ + TR_n \cdot \text{pipeline}(N_n, S(N_n))) \end{aligned} \quad (3)$$

$$\begin{aligned} \text{energy} = \sum_{n=0}^{\text{len}(N)} (e(N_n, s(N_n)) + TR_n \cdot e(N_n, s(N_n), \text{OUT}) \\ + TR_n \cdot e(N_{n+1}, s(N_{n+1}), \text{IN})) \end{aligned} \quad (4)$$

The AxoNN achieves an accuracy of 97.1% on execution time prediction and 98.2% on energy consumption prediction. In the worst case scenario, the accuracy reduces to 78.1% and 71.9%, respectively. As mentioned previously, increasing the number of transitions not only involves an extra overhead of interlayer performance but also requires more time for the solver to suggest the optimal scheduler, going from 5 s for a single transition to 1 min for three transitions on the Jetson AGX Xavier [26]. A comparison between the optimal schedule for a model and the GPU-only execution was absent from this article, as this would vary depending on the ECT set by the user.

3) Genetic Algorithm (GA): Another approach explored in [1] involved using a GA to schedule tasks based on profiling data from the different accelerators. Similar to the AxoNN and HaX-CoNN, the transition and execution time are profiled before

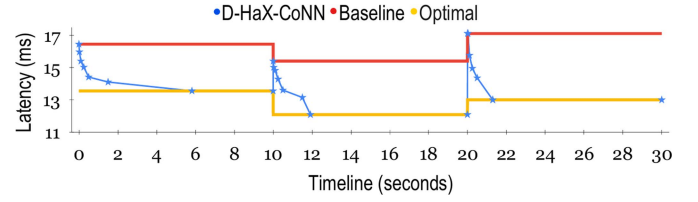


Fig. 3. Dynamic execution where the system changes every 10 seconds [7].

it passes to the GA. The GA then generates the ideal partitioning point for the DNN. The mechanism of the algorithm is similar to evolutionary processes in nature, where only the best-fit chromosome (an array of layers where the index represents the processing element) survives. The approach was initially tested on smartphones like the Galaxy S9 and then later tested on the Xavier device in [27]. The objective of the GA scheduler when using a single DNN is to maximize the throughput and minimize the energy consumption. In the case of multiple DNNs, the objective is to reduce the latency and energy consumption. The GA method assigns layers of the DNNs to the processing elements within the CPU, taking advantage of parallelism within and between the processing units.

4) HaX-CoNN: HaX-CoNN [7] is built on Herald and H2H. The difference is that it uses a constraint-based satisfiability problem to minimize the latency or maximize the throughput (the objective changes depending on the application), as shown by (5). In addition, it considers memory contention caused by the concurrent execution of layers with the use of processor-centric contention-aware slowdown model (PCCS) [24]. This is evident from (6), where the value of $C_{N_n, s(N_n)}$ is found using PCCS.

$$\begin{aligned} \max \sum_{i=0}^{\text{len}(DNN)} \frac{1}{\text{latency}(N, CE, S(N \rightarrow CE))_i} \\ \min \max \text{latency}(N, CE, S(N \rightarrow CE))_i \end{aligned} \quad (5)$$

$$\begin{aligned} \text{latency} = \sum_{n=0}^{\text{len}(N)} (L(N_n, s(N_n)) \times C_{N_n, s(N_n)} \\ + TR_n \cdot \tau(N_{n+1}, s(N_{n+1}), \text{IN}) \\ + TR_n \cdot \tau(N_n, s(N_n), \text{OUT})) \end{aligned} \quad (6)$$

Considering the application, HaX-CoNN can suggest optimal schedules for various types of DNN execution. For two instances of the same DNN running concurrently, it improved throughput up to 29% compared to GPU-only and GPU-DLA execution. For two DNNs running simultaneously on either the same data or streaming data, latency and throughput improved by 23%. The CPU is not considered in executing tasks but rather to carry out the scheduling in the case of the dynamic version (D-HaX-CoNN). D-HaX-CoNN starts with an initial naïve schedule. With this, the concurrent DNN loop begins execution, and its schedule is updated with a more optimized version by the Z3 solver. The solver runs until the schedule can no longer be optimized. Fig. 3 shows the effectiveness of employing the D-HaX-CoNN scheduler, where it can come to the ideal solution

in less than 2 s for the two systems with a pair of DNNs. There are a total of three changes in the graph. Part 1 is where GoogleNet and ResNet152 are run serially, while ResNet18 runs in parallel. Part 2 is where Inception and ResNet152 are run concurrently. Part 3 replaces Inception in the previous case with a visual geometry group (VGG) model, VGG19. In the case of the first part, the system has three DNNs, and therefore more layer groups and possibilities to consider. Moreover, the overhead of running the Z3 solver on the CPU, alongside the concurrent DNN execution in DLA and GPU, is less than 2% [7].

5) Map-and-Conquer: Map-and-Conquer [32] follows a unique approach compared to all the schedulers mentioned earlier. First, it implements parallelism along the horizontal to divide the model into several independent inference stages that can run simultaneously, as opposed to the vertical pipelining (layerwise). The work in [35] argues that the division of layers across accelerators is not the most efficient way to utilize them. Operations, such as convolution and multihead self-attention, can be parallelized so that multiple operations of the same layer are concurrently carried out on different devices to maximize efficiency. Second, it is one of the first works to consider transformer models apart from DNN models, making it more versatile in terms of model architecture. Similar to GA, it uses an evolutionary algorithm to identify the ideal schedule, based on profiling data from TensorRT, and a predictor, XGBoost [36], to provide layer estimates of latency and energy on each of the devices to the algorithm. The predictor is required as it accounts for the computation and communication costs, similar to the PCCS model in HaX-CoNN. For VGG-19, compared to GPU-only execution, which consumes 630 mJ, Map-and-Conquer can provide a scheme with 136-mJ consumption (4.62x reduction) while keeping the latency relatively same (25 ms). The objective function of Map-and-Conquer is given by (7). It allows the user to set an energy consumption threshold (similar to the AxoNN), latency threshold, and the performance objective, thus allowing for greater flexibility in the pipeline creation [32].

$$\min P \text{ s.t. } \text{latency} < LT, \text{energy} < ECT, \text{size}(F, I) < M \quad (7)$$

6) Mapping-Aware Graph Neural Architecture Search (MaGNAS): The mapping-aware graph neural architecture search (MaGNAS) is another scheduler that uses an evolutionary algorithm, but developed for the vision graph neural network (GNN). Compared to GPU-only execution, MaGNAS is able to reduce latency by a factor of 1.57 and increase efficiency by a factor of 3.38 on the Xavier device. It utilizes a two-step evolutionary algorithm to optimize the architecture of the GNN and perform the layer mapping based on the performance modeling. During architecture optimization, the different generations of architecture are evaluated on a fitness function that considers accuracy, latency, and energy. After ranking them, some are eliminated, and the remaining undergo mutation and crossover to get a new generation. The process continues until the search budget is reached. The exact process is carried out to identify the ideal hardware partitioning. Similar to Map-and-Conquer, it allows the user to set an energy consumption and latency threshold [37]. It does not consider the memory contention as part of the fitness function.

IV. COMPARATIVE ANALYSIS

A. Idle Time

CP-CNN seeks to reduce the idle time between the DLA and GPU, unlike Jedi, leading to faster inferences and more energy savings in the long run for streaming data. Fig. 2 illustrate this difference as Jedi has idle instances in which either the DLA or the GPU is unused. Herald and H2H approach the scheduling of concurrent DNNs similarly by seeking to reduce the idle time when splitting the models between the accelerators. This results in a faster inference compared to independent model execution on the accelerators, as seen in LP. Regardless, they are ineffective due to the absence of an optimization framework and omission of memory contention, which is why HaX-CoNN is able to outperform them. These differing in approaches impact the partitioning point of the model. For instance, in Jedi, YOLOv3 is split into DLA and GPU at the 57th layer, whereas in CP-CNN, the split occurs at the 16th layer.

B. Intra-Accelerator Parallelism

Among all the schedulers, Jedi maximizes intrapipelization to the highest degree using multiple threads, streams, and PND. However, finding the ideal combination is not as apparent since it is determined through trial and error. Extensive profiling data must be collected before the final schedule can be decided, as it is not just the layers but also the pipeline, streams, and threads that vary. Only the cut point is obtained through the algorithm. The experiments also prove that the nature of the model impacts these parameters, so each model requires customized pipelines for efficient execution. Nevertheless, it is worth exploring as utilizing the parallelism within and between the accelerators improves the throughput and reduces power consumption compared to GA, as evidenced by Fig. 4. Other schedulers that implement concurrent DNNs, such as Herald, H2H, and HaX-CoNN, could be modified to take advantage of the parallelism within the GPU and DLA to improve the inference efficiency further. Rather than letting the remaining DLA and SMs in the GPU remain idle, adding them to the pipeline would be more advantageous.

C. Energy Consumption

Map-and-Conquer is the ideal choice for energy-sensitive applications as it optimizes the execution of a single model using horizontal scheduling. It is also adaptable in handling multiple model architectures, provided that the layers are DLA compatible. The AxoNN may be used as a workaround for more complex models, where DLA compatibility may be an issue. However, in severe cases, the incompatibility may need to be addressed at the development stage using modifications. Although the AxoNN and MaGNAS also allow users to set an energy threshold, it does not leverage parallelism as one model infers a single frame at a time. Moreover, MaGNAS specifically caters to GNN models. In the remaining schedulers, reduced energy consumption is either an indirect consequence or one of the objective functions of the scheduler. This approach is ill suited for applications with strict power constraints. Case in point, even though CP-CNN considers the computing power in

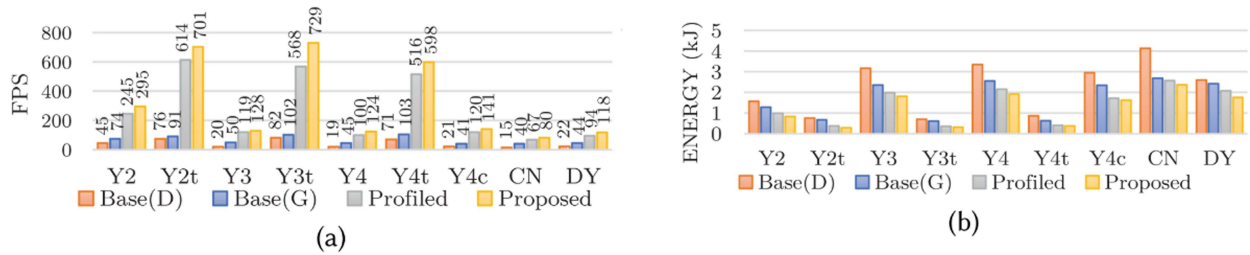


Fig. 4. Throughput and energy consumption of different DNN models when executed on Jetson AGX Xavier using the schedulers. Y refers to the YOLO model, CN refers to CSPNet, and DY refers to DenseNet with YOLO. The performance of Jedi (proposed) is compared with profiled (GA) [1], DLA only [base(D)], and GPU only execution [base(G)]. [27]. (a) FPS. (b) Energy.

its algorithm, it uses the power ratio, not the power value itself, to determine the cut point. It does not ensure that the optimal pipeline consumes the least energy.

D. Transitions and Transition Costs

Generally, most schedulers employ a single data transfer from the DLA to the GPU, as frequent switches between these devices introduce inefficiencies. This has proven to be one of the most efficient methods over constant switching between DLA and GPU. In rare instances, using more than one transition is preferred, like CSPNet in Jedi's case. Even the AxoNN and CP-CNN explore the possibility of implementing more than a single transition. However, the AxoNN opts for a single transition as bottlenecks in the limited bandwidth cause an increased transition time. In CP-CNN, multiple transitions are considered for devices containing more than one GPU. Map-and-Conquer and LP are the only schedulers that do not consider layer partitioning. Map-and-Conquer uses horizontal scheduling so the partitioning is done along the model's width, resulting in subsets that can be independently executed. However, this only allows a single model or frame to be executed. On the other hand, LP executes the entire model on the device, only accounting for the utilization and latency. Over time, such a system can incur latency, potentially violating the application's requirements, as it might require the output in a specific time frame. It is worth noting that transition costs were considered in schedulers developed after 2021, including H2H, AxoNN, CP-CNN, and HaX-CoNN.

E. Memory Contention

The only scheduler that accounts for the memory is the HaX-CoNN scheduler. Adding this parameter to the algorithm improves the system's efficiency and latency prediction. Memory contention is an issue even with a single model due to the concurrent execution of the GPU and DLA with multiple frames. Some schedulers, like AxoNN and MaGNAS, may not require memory contention as there is no concurrency. Only transition costs need to be accounted for. So far, PCCS is the only technique to model contention, so further studies need to be carried out.

F. DLA Use and Layer Compatibility

Most schedulers treat the two DLAs as a single unit, reducing interdevice transitions and optimizing speed. However, in the case of the Jedi scheduler, a PND was introduced, allowing two instances of the same network to run concurrently, with

each instance mapped to each DLA. The typical order of device operation places the DLA first, followed by the GPU, due to the nature of the task. The DLA is well suited for early-stage tasks like feature extraction, while the GPU better handles later layers for specialized tasks like object detection or image recognition. The issue of layer compatibility of the DLA is reflected in the schedulers as well, where Jedi and CP-CNN can avoid this issue by implementing the transition point in the early part of YOLO models, which are otherwise not fully compatible with DLA. In the case of LP, AxoNN, and HaX-CoNN, the profiling stage ensures that the incompatible layers are permanently assigned to the GPU. Since the model's subset is allocated to the DLA for Map-and-Conquer, incompatible layers may disrupt the pipeline and fallback to the GPU. As DNN models become increasingly complex, the layers present at the beginning of the model may not always be DLA compatible. This may include padding in the deconvolution layer, which is present in many CNN-based generative adversarial network models. Therefore, designing a system that can tackle such issues is beneficial by substituting the layers with similar operations before deployment. Current frameworks, like DeepStream and TensorRT, act defensively as they do not handle these issues and instead offload the CNN models as is to the GPU, resulting in pipeline disruptions.

G. Scheduler Methodology and Latency

Schedulers often use a looped structure to organize layer execution according to specific methodologies, with variations in the parameters used for decision making. The Jedi method is distinguished by its multipolicy approach, while other schedulers assess each layer individually or as a group to seek transition points. In contrast, the LP method addresses the task as a whole, leveraging device heterogeneity by allocating different DNNs to each device. Most papers do not state the time required to identify the ideal schedule. This data point may not be as significant with static schedulers since the schedule is generated before the execution cycle. However, this is necessary for dynamic scheduling as the overhead may negatively impact the performance of the models running in parallel. For HaX-CoNN, this is clearly mentioned (see Fig. 3). The overhead of the scheduler is about 2% when running on the CPU core. These data points are absent for dynamic H2H.

H. Performance Analysis

Most schedulers were assessed on DNN models, such as YOLOv3, VGG-19, ResNet, and DenseNet. Such models

TABLE I

COMPARISON OF YOLOV3 EXECUTION ON NVIDIA JETSON AGX XAVIER

Scheduler	Performance metrics		
	Latency (ms)	Throughput (fps)	Energy / Image (mJ)
GPU-only [8], [27]	18.10-20.00	50-55	329-460
Jedi [27]	-	128	340
CP-CNN [8]	12.96	-	306
GA [27]	-	119	400

Note: The energy values for Jedi and GA were calculated approximately by dividing the energy by the number of images used for inference. The results were taken from the mentioned citation.

continue to be used in industrial applications, where edge performance is critical due to limited storage and computational resources. For instance, ResNet models have been used to classify and analyze soil for agricultural purposes [38]. YOLOv3 models have been used in Internet of Things applications, including monitoring and smart camera use [39]. Currently, the schedulers are evaluated on different models, and no standard model has been tested across all of them. Additional experimentation is required to enable consistent benchmarking. Moreover, the code for some of the schedulers is not publicly available, limiting reproducibility and comparative analysis. The following sections present a brief comparison of some schedulers that were tested on the same models.

1) *YOLOv3 for Multiframe Execution*: Table I compares the results for the execution of the YOLOv3 model using Jedi, GA, and CP-CNN. The better performance of Jedi can be attributed to the horizontal parallelization within the accelerators, emphasizing the importance of proper pipelining and the need for multistreaming. However, based on Fig. 4, the increasing throughput across the different models does not cause an increase in power consumption, which is usually not the case. Comparing the performance to CP-CNN, Jedi utilizes more resources with parallelization, causing a slight difference in energy consumption. Since the idle time is reduced in CP-CNN, it offers a more efficient and quicker pipeline. Overall, CP-CNN reduces the energy consumed per image by 62% to 84%, while Jedi reduces the energy consumed for the entire set of images up to 55% [8], [27]. In the case of GA, unlike Jedi, it does not consider the use of multithreading nor multistreaming, which is the cause of its poor performance compared to Jedi, as shown in Table I. Jedi provides a throughput increase of up to 32% and consumes less energy by up to 55%, compared to GA [27].

2) *VGG-19 and ResNet152 for Multi-DNN Execution*: HaX-CoNN was able to outperform its heuristic predecessors, H2H [28] and Herald [33], as evidenced by Table II. The difference in performance between HaX-CoNN, H2H, and Herald arises from the fact that Herald and H2H do not consider memory contention as part of identifying the ideal partitioning point. Although H2H considers transition costs, this is insufficient as the latency is impacted by the overhead incurred from the concurrent execution of DNNs.

TABLE II

COMPARISON OF VGG-19 WITH RESNET152 EXECUTION ON NVIDIA JETSON AGX XAVIER

Scheduler	Performance metrics	
	Latency (ms)	Throughput (fps)
GPU-only [7]	17.05	58
Herald [33]	19.73	50
H2H [28]	16.55	60
HaX-CoNN [7]	13.01	77

Note: All values are taken from the reference [7].

V. DISCUSSION AND FUTURE PROSPECTS

Overall, the general tendency is to develop schedulers that use optimization techniques with a single transition point between the DLA and GPU. Many determine the ideal schedule using the latency, with some including energy as part of their criteria. As numerous applications require more than a single DNN for execution, there is more emphasis in developing schedulers that can handle multiple DNN instances. Among schedulers, HaX-CoNN is remarkably versatile, accommodating diverse scheduling scenarios, and Map-and-Conquer is adaptable in terms of model architecture. Tables III and IV summarize all the schedulers reviewed in this article. The subsequent sections explore challenges that need to be addressed in future works.

A. Exploring PVA and VIC

It is essential to offload tasks or layers, where possible, to the unused PVA and VIC. In most literature, there is no mention of these devices being employed. One paper explored the relationship between energy and memory contention in CPU, GPU, and PVA [18], which could be utilized in schedulers to ensure an efficient use of PVA. A possible pipeline could involve VIC for preprocessing tasks, such as rescaling and image conversion, and PVA for initial convolutional layers or filter operations. However, NVIDIA's frameworks (DeepStream and VPI) limit the use of the PVA to tracking, filtering, and other preprocessing tasks. Using TensorRT, the DNN model's layers can only be assigned to the DLA or GPU [21], [22].

B. Power Estimation

In addition, there is a need to consider the source of the power and energy statistics. It is unclear where some of the schedulers (see [26] and [32]) get the power and energy metrics. Typically, the Tegrastats tool is used to measure power consumption [17]. Despite this, the tool often underestimates the actual power consumption [40], which can impair energy-sensitive systems.

C. Transformer Models

Transformer-based schedulers are being developed as the popularity of LLMs continues to rise. An example of this is the Map-and-Conquer scheme. Although we can picture the concurrent execution with DNN models, with transformer models, this might be harder to achieve with the increasing memory

TABLE III
SUMMARY OF THE SCHEDULERS REVIEWED IN THIS ARTICLE

Reference	Scheduler	Inter/Intra/Both	S/D	Hardware	Execution	Methodology	Algorithm Aim
[31], 2019	LP	Both	S	GPU, DLA, CPU (execute)	Multi-DNN	LP model	Minimize overall latency
[1], 2020	GA	Both	S	GPU, DLA, CPU (process)	Multi-frame/DNN, Vertical	Parallelization within the accelerator and pipelining between the accelerator using a GA	Highest throughput with low energy consumption
[33], 2021	Herald	Inter	S	GPU, DLA	Multi-DNN, Vertical	Profiling followed by design-space exploration	Reduce latency
[28], 2022	H2H	Inter	S	GPU, DLA	Multi-DNN, Vertical	Profiling followed by a set of algorithms to tackle transition cost and latency	Reduce latency and optimize weight locality
[28], 2022	Dynamic H2H	Inter	D	GPU, DLA	Multi-DNN, Vertical	Profiling followed by a set of algorithms to tackle transition cost and latency	Reduce latency and optimize weight locality
[27], 2022	Jedi	Both	S	GPU, DLA, CPU (process)	Multi-frame, Vertical	Parallelization within the accelerator and pipelining between the accelerator	Highest throughput with satisfactory utilization
[26], 2022	AxoNN	Inter	S	GPU, DLA	Single DNN, Vertical	Empirical modeling with Z3 SMT solver	Minimize latency while ensuring energy consumption is within a threshold
[8], 2023	CP-CNN	Inter	S	GPU, DLA, CPU (process)	Multi-frame, Vertical	Parallel processing of sequential streaming data	Time to execute part on DLA = Time to execute part on GPU
[32], 2023	Map-and-Conquer	Inter	S	GPU, DLA, CPU (execute)	Single DNN, Horizontal	Parallel processing of inference stages using evolutionary algorithm	Minimizes latency and energy consumption based on thresholds
[37], 2023	MaGNAS	Inter	S	GPU, DLA	Single GNN, Vertical	Two-step evolutionary algorithm	Minimizes latency and energy consumption based on thresholds
[7], 2024	HaX-CoNN	Inter	S	GPU, DLA	Multi-DNN, Vertical	Profiling followed by Z3 SAT solver	Maximizes utilization and reduces maximum latency
[7], 2024	D-HaX-CoNN	Inter	D	GPU, DLA, CPU (schedule)	Multi-DNN, Vertical	Profiling followed by Z3 SAT solver	Maximizes utilization and reduces maximum latency

Note: S stands for static scheduling and D stands for dynamic scheduling.

TABLE IV
SCHEDULING COSTS CONSIDERED FOR EACH SCHEDULER

Reference	Scheduler	O/H	TC	E	MC
[31], 2019	LP	O	✗	✗	✗
[1], 2020	GA	O	✓	✗	✗
[33], 2021	Herald	H	✗	✗	✗
[28], 2022	H2H	H	✓	✗	✗
[27], 2022	Jedi	H	✓	✗	✗
[26], 2022	AxoNN	O	✓	✓	-
[8], 2023	CP-CNN	H	✓	✗	✗
[32], 2023	Map-and-Conquer	O	✓	✓	✗
[37], 2023	MaGNAS	O	✓	✓	-
[7], 2024	HaX-CoNN	O	✓	✗	✓

Note: O stands for optimization, H for heuristic, TC for transition costs, E for energy consumption, and MC for memory contention.

and computation requirements. In addition, it is unclear whether there is a performance improvement when the model is scheduled vertically rather than horizontally. Further experimentation is required to assess the performance of transformer models in vertical scheduling.

D. Reinforcement Learning for Scheduling on NVIDIA Edge Devices

Another avenue to explore is the use of AI for scheduling. For instance, SCHED² [41] and DL2 [42] are reinforcement learning (RL) techniques used to determine the best use of resources within a GPU data center [11]. Moreover, the work in [43] depicts how RL can be used even in heterogeneous edge devices with a CPU-GPU architecture. It simultaneously tackled multiple preprocessing, inference, and training jobs, outperforming existing heuristic methods. Similarly, RESPECT [12] applies RL in edge TPU devices to schedule DNN models. In this case, the DNN is modeled as a directed graph. The agent then learns to generate near-optimal schedules with minimal overhead. A combination of the two techniques can be applied to the NVIDIA edge devices, allowing more dynamic scheduling to balance multiple inference workloads on the CPU, GPU, and dedicated accelerators. A priority list can also be set, based on the application.

E. Real-Time Deployment in Industrial Applications

Notably, the reviewed schedulers have not been evaluated with live input streams under real-time circumstances.

Therefore, future work should prioritize the assessment in real-time conditions with real data. Other critical aspects for industrial implementation are scalability, security, and fault tolerance. In terms of scalability, these schedulers (except for MaGNAS) can be implemented for any DNN models. For transformer models, as mentioned previously, Map-and-Conquer may be used; however, it may not be possible to implement the scheduling for all transformer model architectures due to limitations of the DLA. Another facet of scalability is the ability to implement multiple models. HaX-CoNN is an ideal choice as it accounts for memory contention. If more than three DNN models are running concurrently, opting for the simplistic LP scheduler instead would be better. Current schedulers did not consider security and fault tolerance as principal design considerations. They are not immune to faults, as memory crashes can disrupt the pipeline without any way to recover. Developing more kernel-based scheduling techniques may be more suitable to circumvent these issues and operate at a minimum performance. Here, the operating system dynamically schedules the AI models through threads. This would mitigate the risk of pipeline elimination from security threats and interruptions, unlike user scheduling, where the scheduling algorithm is separate from the operating system. Kernel-based scheduling would also improve the system's fault tolerance.

F. Summary

Overall, the discussion highlights five key insights that emerge from the review and comparative analysis. First, future schedulers should better utilize the underused on-chip accelerators, such as the PVA and VIC, which can offload pre- and post-processing tasks to reduce GPU and DLA workloads. Second, more accurate methodologies for power estimation are needed, as tools such as Tegrastats tend to underestimate real energy consumption, limiting the reliability of energy-aware optimization. Third, the integration of transformer models introduces new scheduling challenges due to their high memory and computation requirements, calling for further exploration of both vertical and horizontal scheduling strategies. Fourth, RL presents a promising avenue for developing adaptive and dynamic scheduling schemes that can handle heterogeneous workloads efficiently across CPU, GPU, and AI accelerators. Finally, real-time deployment remains a critical gap in the current literature—few schedulers have been tested with live input streams or evaluated for scalability, security, and fault tolerance, all of which are essential for industrial implementation. Tables III and IV summarize the general trends, showing that most schedulers primarily optimize for latency with limited treatment of transition costs, energy, and memory contention.

Ultimately, the choice of the scheduler must align with the requirements of the target application, as illustrated by Fig. 5. As discussed, in the case of multiple DNNs, it is better to opt for HaX-CoNN where possible, or resort to LP. With a single DNN carrying out real-time processing, CP-CNN and Jedi are ideal choices since they allow for multiframe execution. If energy consumption is a concern, Map-and-Conquer is a better option. Currently, Map-and-Conquer is the only choice for transformer

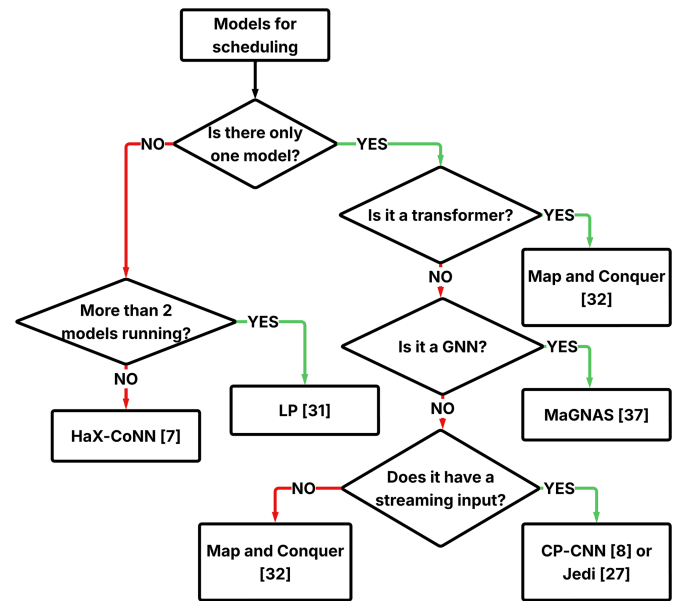


Fig. 5. Flowchart to aid in choosing the ideal scheduler.

models, while for GNN models, MaGNAS is the sole solution for edge devices. These schedulers remain applicable in these devices, even if the architecture changes with new processing units or an updated DLA, as long as the device uses a unified shared memory system.

VI. CONCLUSION

This article systematically compares the different schedulers developed for the more recent NVIDIA Jetson edge devices that include a DLA in its hardware. Some of the research challenges that need to be considered include the further development of the TensorRT layer assignment tools to allocate layers to the CPU and PVA. This can open more possibilities for scheduling and may even be more efficient than just using the GPU and DLA. For successful implementation, this would require studies into the memory contention and latency–energy tradeoffs, similar to [18] and [24], while ensuring that the source of the power statistics is accurate. Exploring layer substitutions to target DLA-incompatible layers within the model would also be beneficial. To improve the security and fault tolerance of the system, it might be more advantageous to combine an RL-based scheduler with the operating system to create a more resilient system. The focus of future scheduling techniques should be to balance multiple AI models as an increasing number of robotics applications demand multimodal systems. Furthermore, the schedulers need to be tested extensively in real-time applications.

REFERENCES

- [1] D. Kang, J. Oh, J. Choi, Y. Yi, and S. Ha, "Scheduling of deep learning applications onto heterogeneous processors in an embedded device," *IEEE Access*, vol. 8, pp. 43980–43991, 2020.
- [2] N. Capodieci, R. Cavicchioli, M. Bertogna, and A. Paramakuru, "Deadline-based scheduling for GPU with preemption support," in *Proc. IEEE Real-Time Syst. Symp.*, Dec. 2018, pp. 119–130.

- [3] H. Xu, S. Huang, Y. Yang, X. Chen, and S. Hu, "Deep learning-based pedestrian detection using RGB images and sparse LiDAR point clouds," *IEEE Trans. Ind. Inform.*, vol. 20, no. 5, pp. 7149–7161, May 2024.
- [4] S. Protasov et al., "CNN-based omnidirectional object detection for HermesBot autonomous delivery robot with preliminary frame classification," in *Proc. 20th Int. Conf. Adv. Robot.*, 2021, pp. 517–522.
- [5] S. Martin, "Saildrone charts autonomous oceanic monitoring—Resources.nvidia.com," 2023. Accessed: 17 Apr. 2025. [Online]. Available: https://resources.nvidia.com/en-us-jetson-success/saildrone-autonomous?lx=XRDs_y&ncid=no-ncid
- [6] Z. Zhang, C. Niu, Z. Zhao, X. Zhang, and X. Chen, "Small object few-shot segmentation for vision-based industrial inspection," *IEEE Trans. Ind. Inform.*, vol. 21, no. 6, pp. 4388–4399, Jun. 2025.
- [7] I. Dagli and M. E. Belviranli, "Shared memory-contention-aware concurrent DNN execution for diversely heterogeneous system-on-chips," in *Proc. 29th ACM SIGPLAN Annu. Symp. Princ. Pract. Parallel Program.*, New York, NY, USA, 2024, pp. 243–256, doi: [10.1145/3627535.3638502](https://doi.org/10.1145/3627535.3638502).
- [8] D. Chun, J. Choi, H.-J. Lee, and H. Kim, "CP-CNN: Computational parallelization of CNN-based object detectors in heterogeneous embedded systems for autonomous driving," *IEEE Access*, vol. 11, pp. 52812–52823, 2023.
- [9] X. Yuan, H. Li, K. Ota, and M. Dong, "Generative inference of large language models in edge computing: An energy efficient approach," in *Proc. Int. Wirel. Commun. Mobile Comput.*, May 2024, pp. 244–249.
- [10] S. Mittal, "A survey on optimized implementation of deep learning models on the NVIDIA Jetson platform," *J. Syst. Architect.*, vol. 97, pp. 428–442, 2019.
- [11] Z. Ye et al., "Deep learning workload scheduling in GPU datacenters: A survey," *ACM Comput. Surv.*, vol. 56, no. 6, Jan. 2024, doi: [10.1145/3638757](https://doi.org/10.1145/3638757).
- [12] J. Yin, Y. Li, D. Robinson, and C. Yu, "RESPECT: Reinforcement learning based edge scheduling on pipelined coral edge TPUs," in *Proc. 60th ACM/IEEE Des. Autom. Conf.*, 2023, pp. 1–6.
- [13] Libre Computer, "Alta AML-A311D-CC," 2024. Accessed: 9 Jul. 2025. [Online]. Available: <https://libre.computer/products/aml-a311d-cc/>
- [14] L. S. Karumbunathan, "NVIDIA Jetson AGX orin series," 2022. Accessed: 17 Apr. 2025. [Online]. Available: <https://www.nvidia.com/content/dam/en-zz/Solutions/gtc/t21/jetson-orin/nvidia-jetson-agx-orin-technical-brief.pdf>
- [15] Jetson Download Center, 2025. Accessed: 17 Apr. 2025. [Online]. Available: <https://developer.nvidia.com/embedded/downloads>
- [16] "Hardware architectural specification—NVDLA documentation," 2014. Accessed: 17 Apr. 2025. [Online]. Available: <https://nvdla.org/hw/v1/hwarch.html>
- [17] "Platform power and performance," 2024. Accessed: 17 Apr. 2025. [Online]. Available: <https://docs.nvidia.com/jetson/archives/r35.5.0/DeveloperGuide/SD/PlatformPowerAndPerformance.html>
- [18] M. A. H. Monil, M. E. Belviranli, S. Lee, J. S. Vetter, and A. D. Malony, "MEPHESTO: Modeling energy-performance in heterogeneous SoCs and their trade-offs," in *Proc. ACM Int. Conf. Parallel Archit. Compilation Techn.*, New York, NY, USA, 2020, pp. 413–425, doi: [10.1145/3410463.3414671](https://doi.org/10.1145/3410463.3414671).
- [19] CUDA, 2025. Accessed: 17 Apr. 2025. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/index.html>
- [20] cuBLAS, 2025. Accessed: 17 Apr. 2025. [Online]. Available: <https://docs.nvidia.com/cuda/cublas/>
- [21] "TensorRT documentation," 2025. Accessed: 17 Apr. 2025. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/latest/index.html>
- [22] "DeepStream documentation," 2025. Accessed: 17 Apr. 2025. [Online]. Available: https://docs.nvidia.com/metropolis/deepstream/dev-guide/text/DS_Overview.html
- [23] "VPI—Vision programming interface documentation," 2024. Accessed: 17 Apr. 2025. [Online]. Available: <https://docs.nvidia.com/vpi/>
- [24] Y. Xu, M. E. Belviranli, X. Shen, and J. Vetter, "PCCS: Processor-centric contention-aware slowdown model for heterogeneous system-on-chips," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchitecture*, New York, NY, USA, 2021, pp. 1282–1295, doi: [10.1145/3466752.3480101](https://doi.org/10.1145/3466752.3480101).
- [25] A. Boroumand et al., "Google neural network models for edge devices: Analyzing and mitigating machine learning inference bottlenecks," in *Proc. 30th Int. Conf. Parallel Archit. Compilation Techn.*, 2024, pp. 159–172, doi: [10.1109/PACT52795.2021.00019](https://doi.org/10.1109/PACT52795.2021.00019).
- [26] I. Dagli, A. Cieslewicz, J. McClurg, and M. E. Belviranli, "AxoNN: Energy-aware execution of neural network inference on multi-accelerator heterogeneous SoCs," in *Proc. 59th ACM/IEEE Des. Autom. Conf.*, New York, NY, USA, 2022, pp. 1069–1074, doi: [10.1145/3489517.3530572](https://doi.org/10.1145/3489517.3530572).
- [27] E. Jeong, J. Kim, and S. Ha, "TensorRT-based framework and optimization methodology for deep learning inference on Jetson boards," *ACM Trans. Embedded Comput. Syst.*, vol. 21, no. 5, Oct. 2022, Art. no. 51.
- [28] X. Zhang, C. Hao, P. Zhou, A. Jones, and J. Hu, "H2H: Heterogeneous model to heterogeneous system mapping with computation and communication awareness," in *Proc. 59th ACM/IEEE Des. Autom. Conf.*, New York, NY, USA, 2022, pp. 601–606, doi: [10.1145/3489517.3530509](https://doi.org/10.1145/3489517.3530509).
- [29] Y.-S. Hsiao et al., "Zhuyi: Perception processing rate estimation for safety in autonomous vehicles," in *Proc. 59th ACM/IEEE Des. Autom. Conf.*, New York, NY, USA, 2022, pp. 289–294, doi: [10.1145/3489517.3530445](https://doi.org/10.1145/3489517.3530445).
- [30] Apollo Baidu Team, "Apollo: Open source autonomous driving," 2017. Accessed: 17 Apr. 2025. [Online]. Available: <https://github.com/ApolloAuto/apollo>
- [31] R. Pujol, H. Tabani, L. Kosmidis, E. Mezzetti, J. Abella, and F. J. Cazorla, "Generating and exploiting deep learning variants to increase heterogeneous resource utilization in the NVIDIA Xavier," in *Proc. 31st Euromicro Conf. Real-Time Syst.*, 2019, pp. 23:1–23:23.
- [32] H. Bouzidi, M. Odema, H. Ouarnoughi, S. Niar, and M. A. Al Faruque, "Map-and-Conquer: Energy-efficient mapping of dynamic neural nets onto heterogeneous MPSoCs," in *Proc. 60th ACM/IEEE Des. Autom. Conf.*, Jul. 2023, pp. 1–6.
- [33] H. Kwon, L. Lai, M. Pellauer, T. Krishna, Y.-H. Chen, and V. Chandra, "Heterogeneous dataflow accelerators for multi-DNN workloads," in *Proc. IEEE Int. Symp. High-Perform. Comput. Architect.*, Mar. 2021, pp. 71–83, doi: [10.1109/HPCA51647.2021.00016](https://doi.org/10.1109/HPCA51647.2021.00016).
- [34] L. Dagum and R. Menon, "OpenMP: An industry standard API for shared-memory programming," *IEEE Comput. Sci. Eng.*, vol. 5, no. 1, pp. 46–55, Jan.–Mar. 1998.
- [35] R. Hadidi, J. Cao, M. S. Ryoo, and H. Kim, "Toward collaborative inferencing of deep neural networks on Internet-of-Things devices," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 4950–4960, Jun. 2020.
- [36] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA, 2016, pp. 785–794, doi: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- [37] M. Odema, H. Bouzidi, H. Ouarnoughi, S. Niar, and M. A. Al Faruque, "MaGNAS: A mapping-aware graph neural architecture search framework for heterogeneous MPSoC deployment," *ACM Trans. Embedded Comput. Syst.*, vol. 22, no. 5, Sep. 2023, Art. no. 108.
- [38] W. Wu, L. Huo, G. Yang, X. Liu, and H. Li, "Research into the application of ResNet in soil: A review," *Agriculture*, vol. 15, no. 6, 2025, Art. no. 661.
- [39] I. S. Leal, E. Saqib, I. Shallari, A. Jantsch, S. Krug, and M. O'Nils, "Waist tightening of CNNs: A case study on tiny YOLOv3 for distributed IoT implementations," in *Proc. Cyber-Phys. Syst. Internet Things Week*, New York, NY, USA, 2023, pp. 241–246, doi: [10.1145/3576914.3587518](https://doi.org/10.1145/3576914.3587518).
- [40] N. Shalavi, A. Khoshsirat, M. Stellini, A. Zanella, and M. Rossi, "Accurate calibration of power measurements from internal power sensors on NVIDIA Jetson devices," in *Proc. IEEE Int. Conf. Edge Comput. Commun.*, 2023, pp. 166–170.
- [41] Y. Luan, X. Chen, H. Zhao, Z. Yang, and Y. Dai, "SCHED²: Scheduling deep learning training via deep reinforcement learning," in *Proc. IEEE Glob. Commun. Conf.*, Dec. 2019, pp. 1–7.
- [42] Y. Peng, Y. Bao, Y. Chen, C. Wu, C. Meng, and W. Lin, "DL2: A deep learning-driven scheduler for deep learning clusters," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 8, pp. 1947–1960, Aug. 2021.
- [43] Y.-D. Lin, Y.-T. Ling, Y.-C. Lai, and D. Sudyana, "Reinforcement learning for AI as a service: CPU-GPU task scheduling for preprocessing, training, and inference tasks," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 4, pp. 3433–3448, Aug. 2025.